Development: Search engine for short-form video content

https://github.com/1407arjun/video-rag-search

Arjun Sivaraman
Siebel School of Computing and Data Science
University of Illinois Urbana-Champaign
Urbana, Illinois, United States
arjun16@illinois.edu

Meenakshi Suresh
Siebel School of Computing and Data Science
University of Illinois Urbana-Champaign
Urbana, Illinois, United States
msuresh4@illinois.edu

1 MOTIVATION

As the global landscape of digital information shifts from static text and HTML towards dynamic, short-form videos like reels, shorts etc., a critical gap in retrieval has emerged. While contemporary social platforms excel at algorithmic discovery and passive consumption, they are fundamentally not very equipped for precise information retrieval. Currently, a significant volume of high-utility data, ranging from instructional content to cultural insights remains “dark data,” unless discovered by the platform’s algorithm, and search being obscured by a reliance on metadata such as titles, captions and hashtags. This system is motivated by the necessity to transform these opaque media files into a structured, searchable knowledge base with efficiency, ensuring that the wealth of information contained within video frames is as accessible and queryable as traditional text-based resources.

2 SIGNIFICANCE AND USE-CASES

The significance of this system lies in its transition from metadata-dependent search to deep content-aware retrieval. This system addresses diverse use-cases across both professional and consumer sectors. In the realm of digital asset management, content creators and marketing teams can leverage the platform to rapidly discover new or similar content, content reuse and perform analytics tasks based on the data offered. For general users, the system enables highly granular queries, such as locating a specific culinary technique or a niche DIY repair method mentioned in passing within a 30-second reel. Furthermore, the tool serves as an essential utility for academic researchers and fact-checkers who require the ability to conduct semantic searches across thousands of video hours to identify trends, verify context, or track the dissemination of information across platforms. Overall, this system bridges a major gap, helping take users from video consumption to personalized and customizable video retrieval. The main functionality offered by the system to the user is the ability to search video content using natural language, which is enabled by understanding their audio, visual content, and on-screen text.

3 SYSTEM OVERVIEW

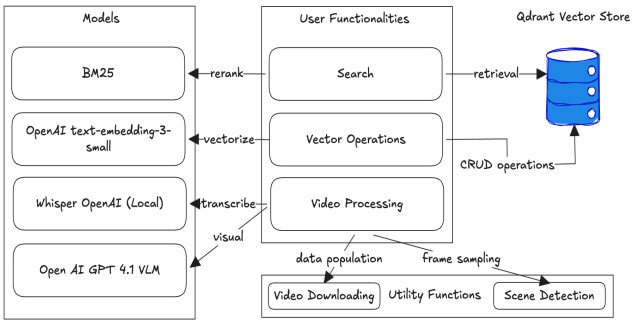

Figure 1: Overview of the system components and their functionalities

Python is used as the preferred programming language to develop this system. FFmpeg is used for all video related operations. The user interface is developed using Streamlit, and the system uses Qdrant as its vector storage and database. The following secrets are used, and have to be located at .streamlit/secrets.toml file at the root:

```
[qdrant]
url = <qdrant_url>
api_key = <qdrant_api_key>

[llm]
endpoint = <hosted_openai_model_endpoint>
api_key = <openai_api_key>
```

This system consists of three main stages to serve as an end to end search engine: the search itself, discovery and ingestion (hereafter referred to as the video processing stage) and data management (hereafter referred to as the vectorization or vector operations stage). Each of the stages is briefly discussed below:

Search: The search functionality is the primary feature of this system. It contains a search bar to query the video library and also an interface which lists relevant videos according to their rank with respect to the search query. The ranking mechanism uses the BM25 model to rank retrieved videos with respect to the search query. This is discussed in detail in section 4.3.

Video Processing: The video processing functionality is essential to populate data in the vector database which is used for search. It accepts a video URL as an input, which is then downloaded for processing using the yt-dlp package. Once downloaded, the video is processed by an ingestion pipeline which contains a series of models (GPT 4.1 for visuals and Whisper for ASR) and utility functions (frame sampling) to capture as much information as possible from the video into text. The extracted text and frames are presented on the user interface to the user for review. The frame sampling and ingestion pipelines are discussed in detail in sections 4.1 and 4.2 respectively.

Vectorization: This stage is present on the user interface as the video library which allows users to view all ingested videos and their metadata that are stored in the vector database and also to delete them from the vector database. Beneath that, it also contains an insertion functionality which is used once a video is processed and reviewed by the user. It uses the text-embedding-3-small model by OpenAI to convert text into embeddings. The metadata is stored alongside its embedding in the vector database and fetched during retrieval. The insertion functionality is discussed in detail along with the video ingestion pipeline in section 4.2.

4 IMPLEMENTATION

In this section, we discuss the essential components of the system in detail including key algorithms and system flows. Although we described the usage of certain models in our system, the system itself is model agnostic and the models can be substituted with any other model from the same class, i.e. Whisper with any other ASR model, a GPT 4.1 with any other VLM, Qdrant with any other vector database, and OpenAI text-embedding-3-small with any other embedding. Hence, in this section, we would be using generic names for all of the models unless specified, for better clarity and understanding.

4.1 FRAME SAMPLING

A video contains multiple frames depending on the fps on which it was shot. This number for example a 30 second video, shot at 30 frames per second is 900 frames. This would mean that the VLM would have to process and generate captions for 900 images for a video that is only 30 seconds long, which is computationally expensive. However, processing all the frames might not even be required in most cases, as a video is a continuous set of frames and these frames share a lot in common between their predecessor and successor frames. This pattern is what we are interested in exploiting in order to select and use only those frames which have a significant change from its predecessor. This approach significantly reduces the number of frames needed to process and supply to the VLM for a majority of the cases.

In our approach to detect changes between frames, we do a pixel-by-pixel comparison between consecutive frames, in order to find the total amount of visual change between the two frames using the Sum of Absolute Differences (SAD) method. There are different kinds of differences between pixels that can be considered for this purpose. The difference that we are targeting is the luminance as scene changes and the human eye are more sensitive to changes in brightness than color, making it a more reliable metric for detecting actual scene cuts rather than minor lighting or color shifts. Hence, each of the frames are converted from RGB to grayscale to facilitate this operation. There is a user-defined threshold (between 0 and 1, currently set to 0.1 in this system). When the total visual change for a frame (normalized to lie between 0 and 1) is greater than the threshold, it means it has enough difference with respect to its previous frame to be considered as a separate frame.

Algorithm: Frame sampling by scene detection
Input: List of all frames in the video (frames), threshold (between 0 and 1)
Output: List of unique frames (final_frames)

```
SET previous_frame_gray = NULL
SET final_frames = [ ]
SET max_difference = video_width * video_height * 255

FOR frame IN frames DO
    // Convert each frame from RGB to grayscale utility
    SET current_frame_gray = convert_rgb_gray(frame)

    // Store first frame in case all frames are the same
    IF previous_frame == NULL THEN
        SET first_frame =  frame
        SET previous_frame_gray = current_frame_gray
        CONTINUE
    END IF

    // Calculate Sum of Absolute Differences (SAD)
    SET total_difference = 0
    FOR i IN frame DO     // Runs for each pixel in frame
        SET diff = abs(current_frame_gray[i] -
                                        previous_frame_gray[i])
        SET total_difference = total_difference + diff
    END LOOP

    // Normalize the score between 0 and 1
```

```
    SET score = total_difference / max_difference

    // Decide to keep frame or discard
    IF score > threshold THEN
       ADD frame TO final_frames
    END IF

    previous_frame_gray = current_frame_gray
  END LOOP

  // Fallback if all frames are the same
  IF size(final_frames) == 0 THEN
     ADD first_frame TO final_frames
  END IF

  RETURN final_frames
END ALGORITHM
```

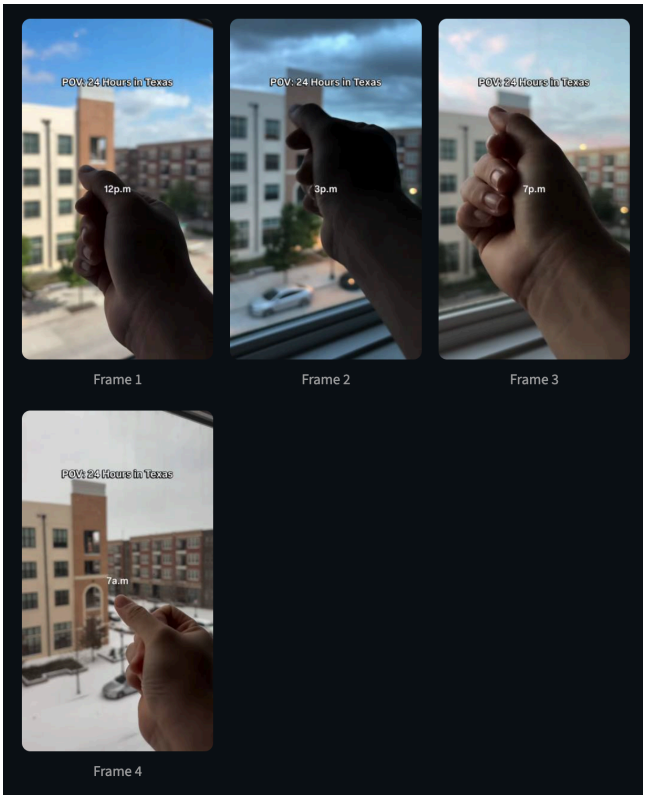

Figure 2: Example of frame sampling in action

4.2 VIDEO INGESTION PIPELINE

The idea is to make a video searchable by converting all its components into text, which can be later converted into text embeddings to compare against user queries. We focus on the audio and the image aspects of the video for extracting information. After the video is downloaded from its source along with its metadata, the ingestion pipeline begins and broadly has three stages - transcription, visual description and vectorization. Figure 3 shows the flow of the ingestion pipeline and the components of each of the stages.

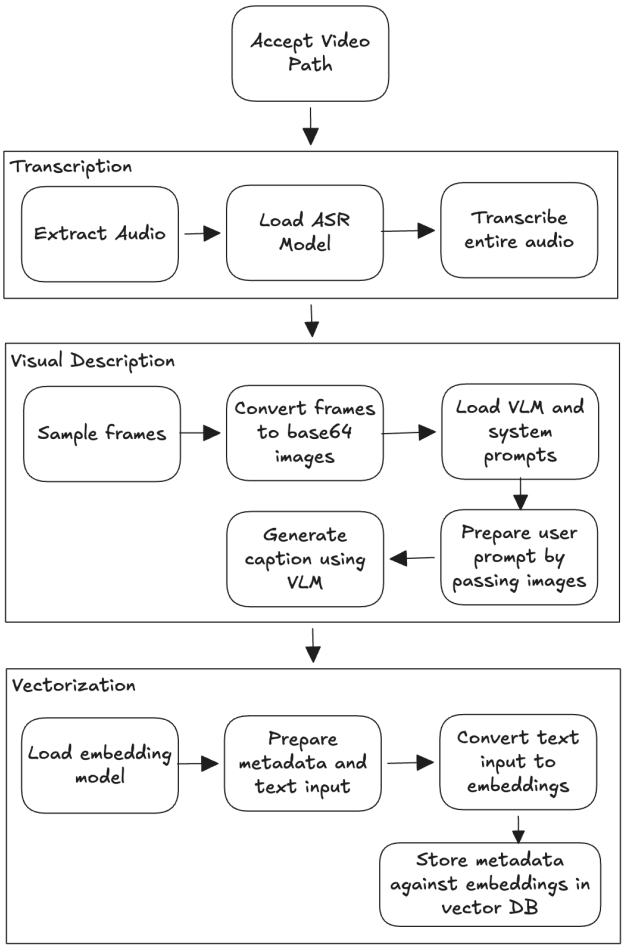

Figure 3: Flow diagram and stages of the video ingestion pipeline

Transcription: This stage deals with the audio aspect of extracting information from the video. The audio is extracted separately from the video using FFmpeg. This audio is stored in a temporary file path. We load an Automatic Speech Recognition (ASR)/Speech-to-Text model which would provide a transcript of the audio. Once the model is loaded, we pass the complete temporary audio file to the model, which then outputs the transcript as a string in the response.

Visual Description: This stage deals with the more complex component of the video i.e. understanding the scene and describing the video content itself. Each video is made of multiple images i.e. frames. Due to reasons explained in section 4.1, we require only a subset of the video's frames in order to extract the complete information of the video. Using the algorithm in section 4.1 and all the

video frames provided for the given video by FFmpeg, we get a list of potential frames for processing and analyzing. Each of these frames are converted into base64 encoded JPEG images using open-cv, and are hereafter referred to as the frames.

A system prompt is designed to instruct the VLM to describe what is visually happening in each frame provided, including key objects, people, actions and on-screen text. We do not need to perform a separate OCR step, as the VLM is already capable of reading text from the frame if any and including it in its description as mentioned in the system prompt. The VLM is then loaded with the system prompt. A user prompt is generated by including each of the frames above as a user message, instructing to describe what is happening in the frame. Finally, both system and user prompts are passed to the VLM, and it generates the visual description of the frames and outputs it as a string.

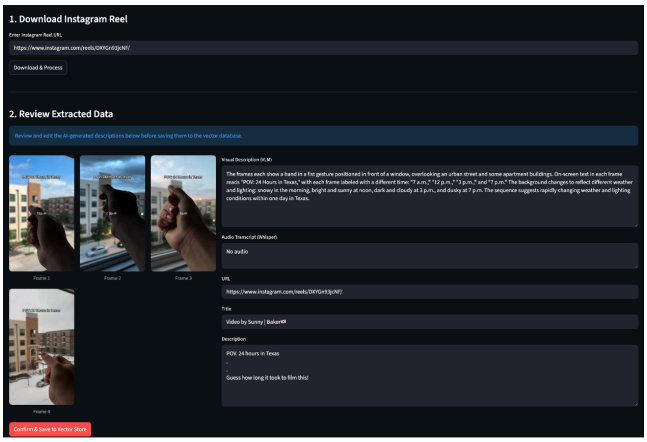

Figure 4: Example of the video ingestion pipeline in action

Vectorization: This stage deals with the preparation of text embeddings and metadata suitable for vector ingestion. Once the transcription and visual descriptions are obtained for the given video along with the original metadata at the time of download, the following text input is created:

Visual Description: <visual-description-text>
Audio Transcript: <transcription-text>
Title: <title-from-original-metadata>
Description: <caption-from-original-metadata>
URL: <download-url>

The following metadata is created against the above text input:

class Metadata(TypedDict):
  transcript: **str**
  visual_description: **str**
  title: **str**
  description: **str**
  thumbnail: **str**
  url: **str**

Here, the first frame of the frames obtained after sampling is selected as the thumbnail. We load an embedding model to vectorize the text input into embeddings. The entire text input block above is passed onto the embedding model whose output is inserted into the vector database along with the generated metadata which is mapped to the embedding. This ensures that whenever embeddings matching a particular query are returned, we are able to provide more information about the returned embeddings in a user conceivable format containing the actual video information.

4.3 RETRIEVAL AND RANKING

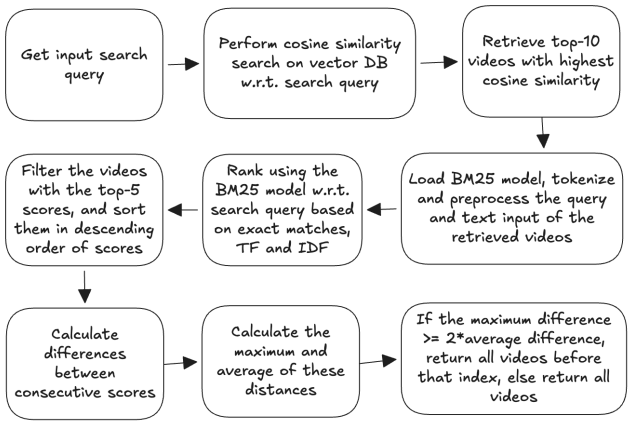

Figure 5: Flow diagram of the search functionality

The search operation is designed as a two step process of retrieval and then ranking in order to increase its relevance and accuracy in getting both relevant videos and at the same time and their ordering to the user. The search returns the top-5 (at most 5) videos with respect to the query.

Retrieval: For a given search query by the user, the system retrieves the top-10 videos from the vector database by performing a cosine-similarity search. The returned videos are the ones having the highest cosine similarity scores with respect to the search query.

Ranking: In the previous step, we retrieve more videos than we intend to return. Vector search is great at finding the semantic meaning (e.g., matching “dog” to “puppy”), but it sometimes fails at exact keyword matching (e.g., a specific brand name or recipe ingredient). On the other hand, a lexical model like BM25 does not capture semantics but has the ability to score videos based on keyword appearances. This allows the model to rank exact keyword matches higher than only semantic matches. Hence, we use a combination of these two strategies to get a desirable ranking result. The vector search allows us to find

semantically matching videos since words in the search query might not exactly match with our video contents. In order to enhance the ranking, we preprocess the search query and text input of the retrieved videos by converting them into lowercase, removing punctuation and stop words and lemmatization (since we have already captured the semantic meaning using vector search first). Once we have overcome the semantics and the preprocessing, the BM25 ranks the videos based on how closely their contents match to the original search query.

At the same time, there can be videos that are retrieved and are far less relevant than the ones which are actually relevant i.e. there is a disparity in scores. For example, videos with scores 5.3, 4.9, 4.2, 0.6, and 0.3. Here, the scores of the last two videos are exceptionally smaller than the scores returned for the top results. In this case, by keeping a disparity threshold we can eliminate the last two videos from the final results and return only 3 results instead of 5. We calculate the differences between consecutive scores and the maximum and average of these differences. If the maximum difference is greater than or equal to 2 times the average difference, then we decide to limit the result to all videos before the maximum difference point, else we return the original result. Thus, the above discussed approach combined gives us the best of both worlds, and improves the search experience and accuracy for the user.

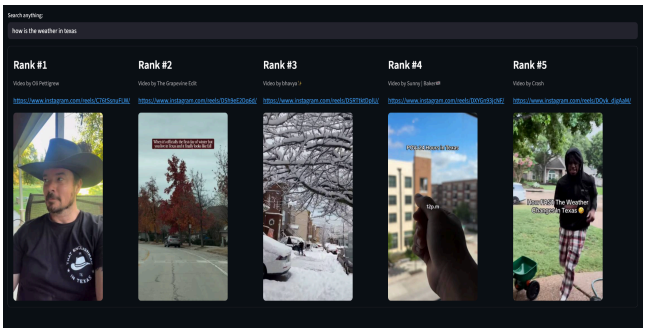

Figure 6: Example of the search in action

5 EVALUATIONS

We evaluated the system on returning the top-5 results for a query. We calculated the precision@k, recall@k, F1-score@k metrics for a data of 50 curated queries and 20 videos. The queries are a manually curated mixture of direct keyword matches and semantic matches corresponding to one or more videos. We tested our system against the existing search systems which are based on only captions and a system which uses captions and transcript together. Following are the results for K = 1, 3 and 5:

Table 1: Evaluation scores at K = 1

Search with	precision@1	recall@1	F1-score@1
Caption only (existing systems)	0.6346	0.5247	0.5526
Caption + transcript	0.6923	0.5856	0.6135
Current system	0.9038	0.7535	0.7897

Table 2: Evaluation scores at K = 3

Search with	precision@3	recall@3	F1-score@3
Caption only (existing systems)	0.6346	0.6119	0.6207
Caption + transcript	0.7179	0.6952	0.7040
Current system	0.9295	0.9026	0.9130

Table 3: Evaluation scores at K = 5

Search with	precision@5	recall@5	F1-score@5
Caption only (existing systems)	0.6667	0.6667	0.6667
Caption + transcript	0.7247	0.7247	0.7247
Current system	0.9349	0.9349	0.9349

6 FUTURE IMPROVEMENTS

Based on the evaluations above, the current system could successfully enable natural language search on short-form video contents. However, the current system can be improved in both user experience as well as performance.

Better Frame Sampling: Instead of comparing pixel-by-pixel, we compare the overall color distribution. We convert the image to the HSV color space and calculate the histogram. If the color palette shifts drastically, it's a true scene change. Another improvement could be, instead of a hardcoded 0.3 threshold, calculating a rolling average of the SAD scores over the last 30 frames and then triggering a scene cut only when the current frame's score spikes significantly higher than the local average.

Categorization: Automatically generate categories based on uploaded videos or infer from hashtags. Each video would be assigned one or more categories. If a video does not fit any category, then a new category would be generated based on its contents. These categories could be clustered to enable searching by category.

Advanced Search: Instead of just returning the full 30 or 60-second video, the system could store chunked embeddings for every 5-second interval. When a user searches for "adding the garlic," or any other specific step/information, the system would return the timestamp range where that action occurs, jumping the user directly to that part in the video.

Discovery: Currently the system supports ingestion from Instagram Reels only. We can expand the download logic to support TikTok, YouTube Shorts, and other short-video platforms, creating a unified cross-platform content library. Also, instead of manual URL entry, the system could be integrated with the Instagram Graph API or TikTok API via webhooks to automatically ingest and index new reels the moment a monitored account posts them.

User Feedback: We can add thumbs up/thumbs down buttons to search results. This user interaction data could be used to adjust the weightings in the BM25 reranker, creating a system that learns and improves its search relevance over time based on what specific items the user actually clicks on, creating a feedback loop.

CONTRIBUTIONS

The system was developed collaboratively by Arjun Sivaraman and Meenakshi Suresh, with each member contributing to different core components of the system. The work was divided to ensure efficient development across data discovery, data processing, retrieval, integrations, and evaluation tasks. Arjun contributed to the development of the video ingestion pipeline, embeddings, integration of the vector database, and implementation of the Streamlit-based user interface for the system. Meenakshi contributed to integrate the functionality of video retrieval from source platforms and data discovery, development of the search infrastructure involving retrieval and ranking components, and conducting system evaluations and performance analysis. Overall, both members played complementary roles that were essential to the successful design, implementation, and evaluation of the project.